# CS 301: High-Performance Computing
# Lab 01 - CPU architecture, Triad and Measuring Performance

Shobhitchandra Chaudari (202301403)
Vraj Parikh (202301440)

February 3, 2026

# Contents


**1 Introduction** **3**

**2 Hardware Details** **3**
2.1 Hardware Details for LAB207 PCs . . . . . 3
2.2 Hardware Details for HPC Cluster (Node gics0) . . . . . 3

**3 Problem Description** **3**

**4 Benchmarking Methodology** **4**

**5 Graphical Results** **4**
5.1 Vector Copy Operation: $a[i] = b[i]$ . . . . . 4
5.2 Vector Scaling Operation: $a[i] = k * b[i]$ . . . . . 5
5.3 Vector Sum Operation: $a[i] = b[i] + c[i]$ . . . . . 5
5.4 Vector Triad Operation: $a[i] = b[i] + c[i] * d[i]$ . . . . . 5

**6 Analysis Task** **5**

**7 Conclusion** **5**

# 1 Introduction

In this lab, we analyze CPU performance on both a laboratory workstation and an HPC cluster. The focus is on understanding how memory hierarchy and arithmetic operations affect execution time by implementing basic vector kernels such as copy, scale, sum, and triad. These kernels are widely used in performance benchmarking and provide insight into memory bandwidth limitations.

# 2 Hardware Details

## 2.1 Hardware Details for LAB207 PCs

- **Architecture:** x86_64
- **CPU(s):** 12
- **Core(s) per socket:** 6
- **Thread(s) per core:** 2
- **Model name:** 12th Gen Intel(R) Core(TM) i5-12500
- **L1 Cache:** 480 KB
- **L2 Cache:** 7.5 MB
- **L3 Cache:** 18 MB

## 2.2 Hardware Details for HPC Cluster (Node gics0)

- **Architecture:** x86_64
- **CPU(s):** 24
- **Socket(s):** 2
- **Core(s) per socket:** 6
- **Model name:** Intel(R) Xeon(R) CPU E5-2620 v3
- **L1 Cache:** 64 KB per core
- **L2 Cache:** 256 KB per core
- **L3 Cache:** 15 MB

# 3 Problem Description

The objective of this experiment is to evaluate system performance using STREAM-like vector benchmarks. The operations involve sequential memory accesses and simple arithmetic, making them ideal for studying the impact of cache size and memory bandwidth on execution time

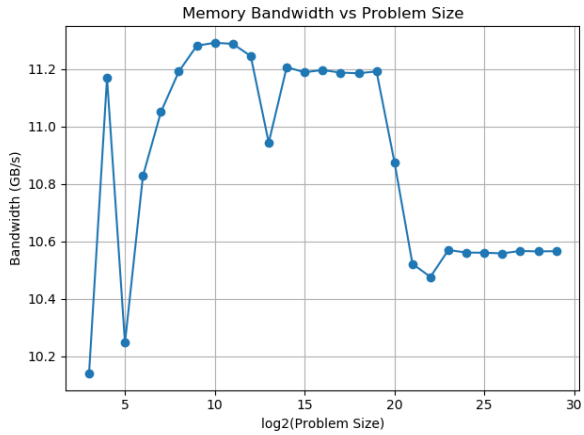


Figure 1: Bandwidth vs. Problem Size for the vector copy operation

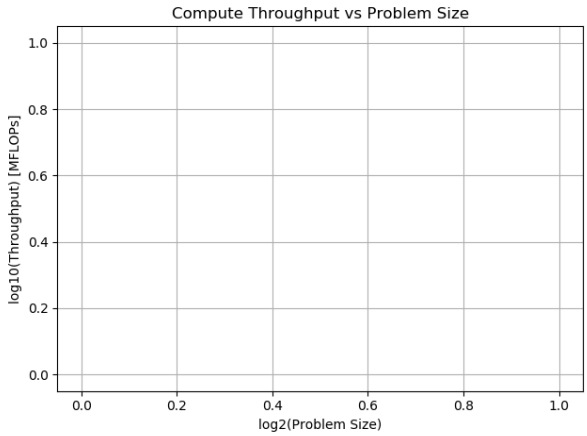


Figure 2: Throughput vs. Problem Size for the vector copy operation

# 4 Benchmarking Methodology

Each kernel was executed for varying problem sizes while keeping the total amount of work constant. Execution time was measured using `clock_gettime()` with the `CLOCK_MONOTONIC` clock. Memory bandwidth and computational throughput were calculated using:

$$Bandwidth(GB/s) = \frac{Bytes\ per\ element \times N_p \times RUNS}{T_{algo} \times 10^9}$$

$$Throughput(MFLOPs) = \frac{FLOPs\ per\ element \times N_p \times RUNS}{T_{algo} \times 2^{20}}$$

# 5 Graphical Results

This section presents the performance results obtained from running the vector kernels on both platforms

## 5.1 Vector Copy Operation: $a[i] = b[i]$

The copy operation involves one read and one write per element and does not include any arithmetic computation. Its performance is entirely limited by memory bandwidth.

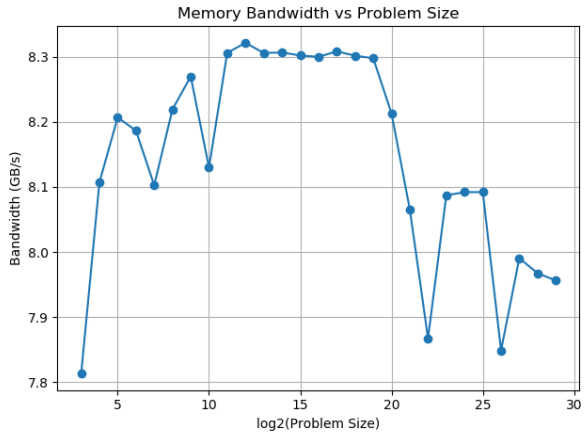


Figure 3: Bandwidth vs. Problem Size for the vector scaling operation

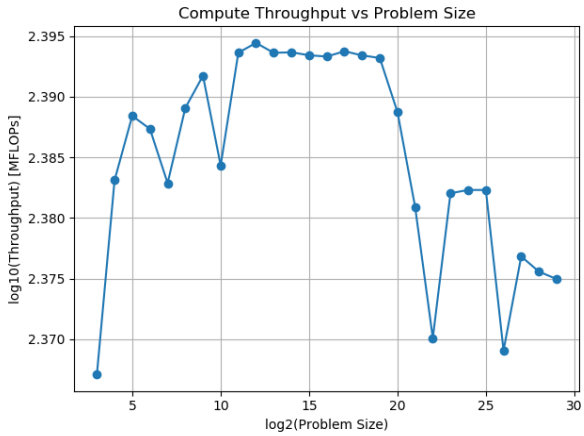


Figure 4: Throughput vs. Problem Size for the vector scaling operation

### 5.2 Vector Scaling Operation: $a[i] = k * b[i]$

### 5.3 Vector Sum Operation: $a[i] = b[i] + c[i]$

### 5.4 Vector Triad Operation: $a[i] = b[i] + c[i] * d[i]$

## 6 Analysis Task

The results indicate that performance is strongly influenced by memory hierarchy. High throughput is achieved when data fits within cache, while a decline occurs once accesses are redirected to main memory.

## 7 Conclusion

This experiment highlights the importance of memory bandwidth in determining application performance. Understanding these limitations is essential for designing efficient high-performance computing applications.

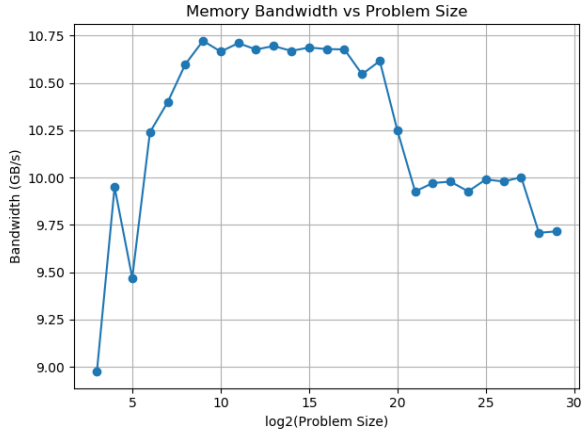


Figure 5: Bandwidth vs. Problem Size for the vector add operation

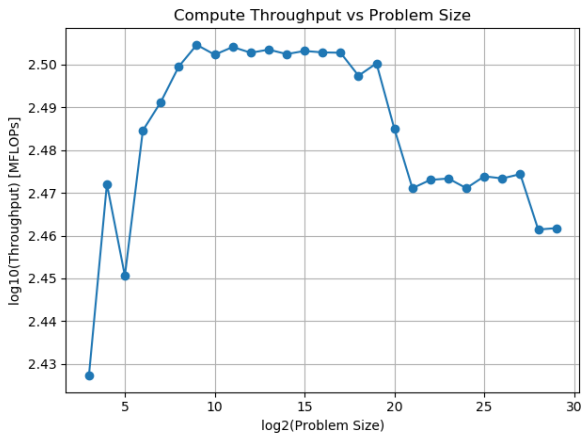


Figure 6: Throughput vs. Problem Size for the vector add operation

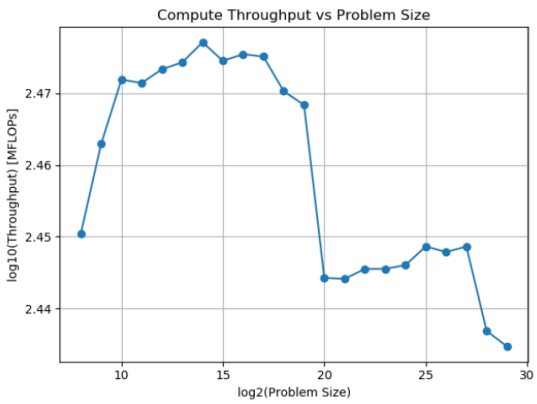


Figure 7: Throughput vs. Problem Size for the vector triad operation

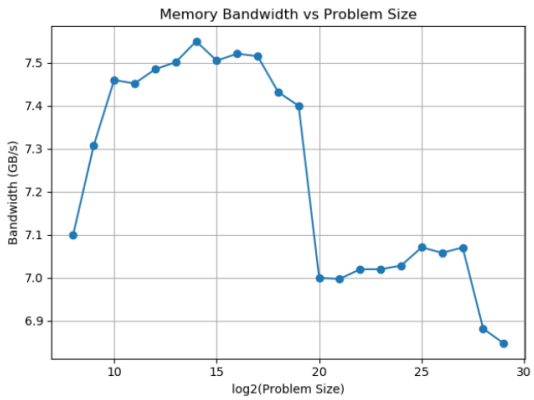


Figure 8: Bandwidth vs. Problem Size for the vector triad operation